

HELIX_SCRIPT_GUIDE

Helix Script Command Guide



Complete reference for the HelixCode facade script (helix) with all commands, options, and usage examples.

Quick Start

Basic Usage

```
# Make script executable
chmod +x helix
```

```
# Show help
./helix help
```

```
# Start all services
./helix start
```

```
# Check status
./helix status
```

```
# Run CLI commands
./helix cli --help
```



```
# Open Terminal UI
./helix tui
```

Command Reference

./helix start

Start all HelixCode containers and services.

Usage:

```
./helix start
```

What it does: - Starts main HelixCode container - Starts PostgreSQL database - Starts Redis cache - Starts worker nodes - Configures network and ports - Displays connection information

Auto-port behavior: - If `HELIX_AUTO_PORT=true` (default), automatically finds available ports - Adjusts ports if default ports (8080, 2222, 3000) are occupied

`./helix stop`

Stop all HelixCode containers.

Usage:

```
./helix stop
```

What it does: - Stops all running containers - Preserves data volumes - Clean shutdown

`./helix restart`

Restart all HelixCode containers.

Usage:

```
./helix restart
```

What it does: - Stops all containers - Starts all containers - Equivalent to `./helix stop && ./helix start`

`./helix status`

Show container status and connection information.

Usage:

```
./helix status
```

What it displays: - Container running status - Service URLs and ports - Accessible directories - Quick command reference

`./helix logs`

View container logs.

Usage:

```
# View all logs
./helix logs
```

```
# View specific service logs
```

```
./helix logs helixcode
```

```
./helix logs postgres
```

```
./helix logs redis
```

```
./helix logs worker-1
```

```
# Follow logs (real-time)
```

```
./helix logs -f
```

```
# View last N lines
```

```
./helix logs --tail=100
```

./helix cli

Execute CLI commands in the container.

Usage:

```
# Show CLI help
```

```
./helix cli --help
```

```
# Health check
```

```
./helix cli --health
```

```
# List workers
```

```
./helix cli --list-workers
```

```
# List models
```

```
./helix cli --list-models
```

```
# Generate with LLM
```

```
./helix cli --prompt "Hello world" --model llama-3-8b
```

```
# Add worker
```

```
./helix cli --worker "192.168.1.100" --user "developer" --key "/  
path/to/key"
```

```
# Send notification
```

```
./helix cli --notify "Task completed" --notify-type "success"
```

Auto-start behavior: - Automatically starts container if not running - No need to run `./helix start` first

./helix tui

Start the Terminal UI.

Usage:

```
./helix tui
```

What it does: - Opens the rich terminal interface - Provides visual system management - Shows real-time statistics

Auto-start behavior: - Automatically starts container if not running

./helix server

Start the REST API server.

Usage:

```
./helix server
```

What it does: - Starts the REST API service - Makes API available on configured port

Auto-start behavior: - Automatically starts container if not running

./helix exec

Execute arbitrary commands in the container.

Usage:

```
# Access shell
```

```
./helix exec /bin/bash
```

```
# Run specific command
```

```
./helix exec ls -la
```

```
./helix exec ps aux
```

```
./helix exec curl http://localhost:8080/health
```

```
# Interactive session
```

```
./helix exec -it /bin/bash
```

./helix help

Show help information.

Usage:

```
./helix help
./helix --help
./helix -h
```

Advanced Usage

Environment Variables

The script reads from .env file in the project root:

```
# Port Configuration
HELIX_API_PORT=8080      # REST API port
HELIX_SSH_PORT=2222     # Worker SSH port
HELIX_WEB_PORT=3000     # Web interface port

# Network Mode
HELIX_NETWORK_MODE=standalone # standalone or distributed
HELIX_AUTO_PORT=true         # Auto-adjust ports if occupied

# Security (REQUIRED)
HELIX_DATABASE_PASSWORD=your-secure-password
HELIX_AUTH_JWT_SECRET=your-jwt-secret
HELIX_REDIS_PASSWORD=your-redis-password
```

Auto-Start Feature

Commands that automatically start containers: - ./helix cli - ./helix tui -
./helix server

Example workflow:

```
# No need to start manually!
./helix cli --health      # Auto-starts container
./helix tui               # Auto-starts container
./helix cli --list-workers # Auto-starts container
```

Port Management

When HELIX_AUTO_PORT=true (default): - If port 8080 is occupied, uses 8081, then 8082, etc. - Same for SSH (2222) and Web (3000) ports - Displays actual ports used after startup

Manual port override:



```
HELIX_API_PORT=9090 HELIX_SSH_PORT=2323 ./helix start
```

Common Workflows

Development Workflow

```
# Start development session
./helix start

# Check everything is working
./helix cli --health
./helix cli --list-workers

# Open Terminal UI for monitoring
./helix tui

# Access REST API
curl http://localhost:8080/health

# Stop when done
./helix stop
```

Quick Testing Workflow

```
# No manual start needed!
./helix cli --health           # Auto-starts and runs health check
./helix cli --list-models      # Auto-starts and lists models
./helix tui                    # Auto-starts and opens UI
```

Production Workflow

```
# Start with specific configuration
HELIX_NETWORK_MODE=distributed ./helix start

# Monitor logs
./helix logs -f

# Check status
./helix status

# Graceful shutdown
./helix stop
```

Troubleshooting

Common Issues

Script not executable:

```
chmod +x helix
```

Port conflicts:

```
# Check what's using ports
netstat -tulpn | grep :8080
```

```
# Use auto-port adjustment
HELIX_AUTO_PORT=true ./helix start
```

Container not starting:

```
# Check logs
./helix logs
```

```
# Check Docker daemon
docker info
```

```
# Check resources
docker system df
```

Auto-start not working:

```
# Manual start
./helix start
```

```
# Then use commands
./helix cli --help
```

Debug Mode

Enable verbose output:

```
# Set debug environment variable
HELIX_DEBUG=1 ./helix start
```

Directory Structure

```
helix_code/
├── helix                # Main facade script
```

```

├── .env                # Environment configuration
├── workspace/         # Working directory (mounted)
├── projects/          # Project storage (mounted)
├── shared/            # Shared configuration (mounted)
├── helix_code/        # Source code
└── config/            # Application configuration
    ├── assets/        # Static assets
    └── test/ssh-keys/ # SSH keys for workers

```

Security Notes

- .env file is gitignored for security
- Change default passwords in production
- Use HTTPS in production environments
- Restrict network access to necessary ports only

Quick Reference

Essential Commands

```

./helix start      # Start everything
./helix status     # Check status
./helix cli --health # Health check (auto-starts)
./helix tui        # Terminal UI (auto-starts)
./helix logs       # View logs
./helix stop       # Stop everything

```

Common CLI Commands

```

./helix cli --list-workers # List workers
./helix cli --list-models  # List models
./helix cli --prompt "text" # Generate with AI
./helix cli --notify "msg"  # Send notification

```

Service URLs (After Startup)

```

REST API:      http://localhost:8080
SSH Workers:   localhost:2222
Web Interface: http://localhost:3000

```

Related Documentation

- [DOCKER SETUP.md](#) - Complete Docker setup guide

- [README DOCKER.md](#) - Quick start reference
- [GETTING STARTED.html](#) - Web documentation

This guide provides complete coverage of the `helix` script functionality for easy reference and troubleshooting.